

Chapter 3: Unsupervised Learning

Universidad de Salamanca

Faculty of Sciences

Advanced Programming

Diego M. Jiménez-Bravo · dmjimenez@usal.es

Outline

- 1 Introduction
- 2 3.1 Clustering
- 3 3.2 Dimensionality reduction
- 4 3.3 Anomaly detection

Outline

1 Introduction

2 3.1 Clustering

3 3.2 Dimensionality reduction

4 3.3 Anomaly detection

What is unsupervised learning?

Unlike the previous chapters, here training is **not guided** by any label or target variable. The system receives no “ground truth”; instead, it must explore the intrinsic mathematical relationships in the data to reveal its hidden structure.

Technically, while supervised learning seeks to approximate the relationship $X \rightarrow y$, unsupervised learning works “blindly” to discover the **latent structure** of the data X itself, with no correct reference answer.

As computer scientist Yann LeCun put it: *“If intelligence is a cake, unsupervised learning is the cake, supervised learning is the icing, and reinforcement learning is the cherry.”* The vast majority of real-world data is unlabeled, which makes this paradigm a fundamental pillar of machine learning.

There are three main families of techniques: **clustering** (grouping similar data), **dimensionality reduction** (simplifying complex data), and **anomaly detection** (identifying rare cases).

Practical applications

- **Market segmentation:** grouping customers into homogeneous niches based on demographic and consumption data, without defining the categories in advance, to target specific campaigns.
- **Visualizing complex data:** projecting datasets with hundreds of variables into 2D or 3D representations, preserving their structure, so a person can understand them at a glance.
- **Recommendation systems:** grouping users with similar tastes or products with similar characteristics to predict implicit preferences.
- **Anomaly detection:** modeling the “normal” behavior of data to automatically identify financial fraud or manufacturing-line failures.

Example: an online store that groups its customers by purchasing habits (without being told in advance which groups to look for) in order to design different offers for each group — that’s clustering applied to market segmentation.

Outline

1 Introduction

2 3.1 Clustering

3 3.2 Dimensionality reduction

4 3.3 Anomaly detection

What is clustering?

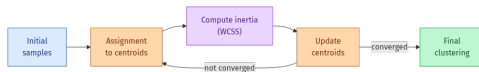
Clustering is the most widespread technique in unsupervised learning. Its goal is to **partition** a dataset into subgroups whose members share **high internal similarity** while, at the same time, showing **high differentiation** from members of other subgroups.

Different families of algorithms exist, depending on how they define group membership: **centroid**-based (K -means), **hierarchy**-based (hierarchical clustering), and **density**-based (DBSCAN). We'll study each in turn.

K-means: the algorithm's mechanics

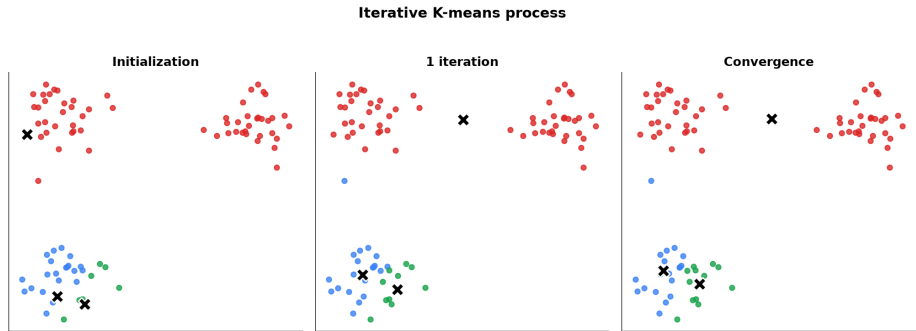
K-means is a **centroid**-based clustering algorithm: it seeks to partition a set of observations into K distinct clusters, where each centroid acts as the “center of gravity” of its group. Its mathematical goal is to minimize the **within-cluster inertia** (WCSS, Within-cluster Sum of Squares). The process repeats iteratively:

- ➊ **Initialization**: K random points are chosen as initial centroids.
- ➋ **Assignment**: each sample joins the nearest centroid (typically by Euclidean distance).
- ➌ **Update**: each centroid is recomputed as the mean of all samples assigned to its group.
- ➍ Steps 2 and 3 repeat until the centroids stop moving: **convergence**.



K-means: the process, visually

The figure below shows the three stages on a dataset with three natural groups: the centroids (marked with an X) start from random positions and converge toward the center of each group after just a few iterations.



Example: it's like seating guests at a party: each guest sits at the nearest table; then each table recenters on its guests; and this repeats until no one needs to move.

K -means++: the initialization problem

K -means is a **heuristic** algorithm and therefore highly sensitive to the random initialization of its centroids. If the initial centroids land in unfavorable regions, the algorithm can converge to a **suboptimal local minimum** — a clustering that looks good locally but isn't the best possible. To mitigate this, K -means++ initialization is used:

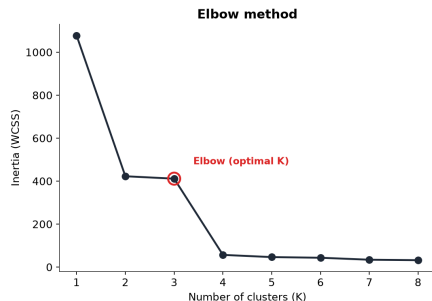
- The first centroid is chosen at random among the data points.
- Each subsequent centroid is chosen probabilistically, giving higher probability to points that are **farther** from the centroids already chosen.

This “smart seeding” ensures the initial centroids are well spread across the space, speeding up convergence and improving the quality of the final result.

The elbow method

Since the number of clusters K is a hyperparameter that must be fixed before training, we need a criterion for choosing it. The **elbow method** plots the inertia (WCSS) against different values of K :

- As K increases, inertia naturally decreases (groups become smaller and more compact).
- The optimal K sits where the curve stops dropping sharply and flattens out: the “elbow”.



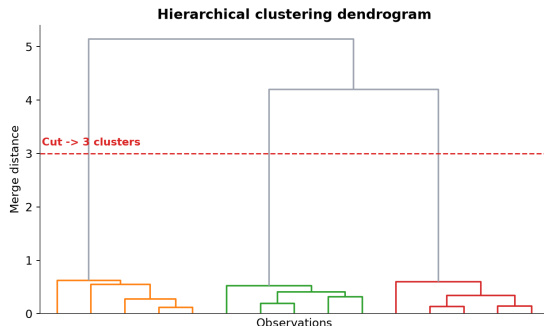
Hierarchical clustering: the agglomerative approach

Hierarchical clustering groups data **without needing to fix** K in advance, and produces a hierarchical structure that is easy to visualize. The most common method is **agglomerative** (bottom-up):

- ➊ Start by treating each individual observation as its own cluster.
- ➋ At each step, compute the distances between all clusters and **merge the two closest ones**.
- ➌ Repeat until all samples are merged into a single, global cluster.

Dendrograms and interpreting cuts

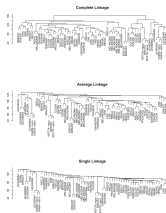
The hierarchy of merges is represented graphically as a **dendrogram**: a tree diagram where the vertical axis shows the distance (or dissimilarity) at which each pair of groups was merged. The user can choose the final number of clusters by making a **horizontal cut** at a given height: the number of lines the cut crosses is the number of resulting groups. This offers a flexibility that rigid algorithms like *K*-means lack.



Linkage criteria

To decide which clusters to merge at each step, we need to define how to measure the distance between groups (not between individual points):

- **Complete linkage**: the **maximum** distance between any pair of points from the two groups. Produces compact, well-balanced clusters.
- **Single linkage**: the **minimum** distance between any pair. Sensitive to noise, and can produce a “chaining” effect (elongated, diffuse groups).
- **Average linkage**: the average of **all** distances between points in both groups. Balances the stability of complete linkage with the tolerance of single linkage.



DBSCAN: fundamentals and parameters

The **DBSCAN** algorithm (Density-Based Spatial Clustering of Applications with Noise) defines groups based on the **local density** of the data, rather than closeness to a centroid or hierarchy. This lets it discover clusters of arbitrary geometric shapes and isolate noise naturally.

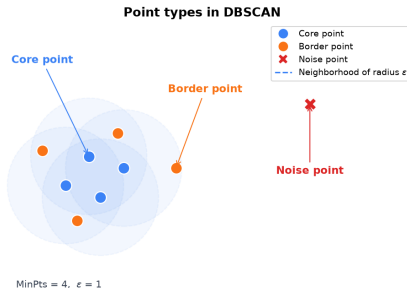
It requires two hyperparameters:

- ϵ (**epsilon**): the neighborhood radius around any point.
- **MinPts**: the minimum number of points required within that radius for the region to be considered “dense”.

DBSCAN: classifying points

During execution, DBSCAN classifies every point into one of three categories, based on the density of its neighborhood:

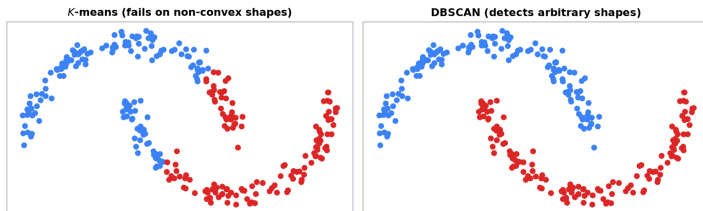
- **Core points:** their ϵ -radius neighborhood contains at least MinPts samples.
- **Border points:** they don't reach MinPts on their own, but lie within the neighborhood of a core point.
- **Noise points:** neither core nor border — considered anomalies and not assigned to any cluster.



DBSCAN: advantages over K -means

- **Robustness to noise:** K -means forces every sample into a cluster (artificially shifting centroids in the presence of outliers); DBSCAN identifies and isolates noise natively.
- **Geometric flexibility:** it doesn't assume clusters must be spherical, so it can discover elongated, nested, or irregular shapes.

K -means vs. DBSCAN on moon-shaped data

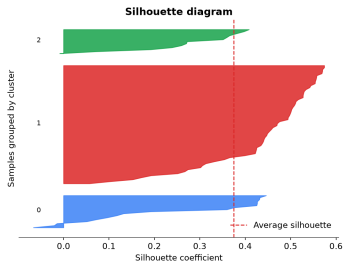


Evaluation: the silhouette coefficient

Without labels, how do we know if a clustering is good? The **silhouette coefficient** compares, for each point, how close it is to its own group versus how close it is to the nearest neighboring group:

$$sc(i) = \frac{b_i - a_i}{\max(a_i, b_i)}$$

The value ranges from -1 to 1 : near $+1$ indicates a well-grouped point, far from other clusters; near 0 , a point on the boundary between two groups; and negative suggests the point is misassigned.



Evaluation: the gap statistic

The **gap statistic** is another way to choose the optimal number of clusters, more rigorous than the elbow method: it compares the inertia observed in the real data with the inertia we'd expect if the data had no clustering structure at all.

Its great advantage: it can detect when the correct answer is that there is **no real grouping** in the data at all — something the elbow method or the silhouette coefficient don't always make clear.

There are more ways to evaluate a clustering, especially when probabilistic models are used:

- **AIC and BIC:** compare models by penalizing the ones with too many parameters, so we don't always pick the most complex one.
- **Cophenetic correlation:** checks whether a dendrogram faithfully reflects the real distances between the data.

In practice, these criteria are rarely computed by hand: machine learning libraries apply them automatically to compare models.

External evaluation with labeled data

When reference labels are available (even if they weren't used for training), we can check how well the discovered groups match the real categories:

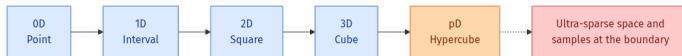
- **Purity**: each cluster is assigned its most frequent label, and we measure the proportion of correct assignments obtained this way. Its weakness: it doesn't penalize having too many small clusters.
- **Adjusted Rand Index (ARI)**: checks whether the model grouped the samples similarly to how the real classes would, correcting for the result we'd expect from pure chance.

Outline

- 1 Introduction
- 2 3.1 Clustering
- 3 3.2 Dimensionality reduction**
- 4 3.3 Anomaly detection

The curse of dimensionality

Modern data analytics often involves datasets with hundreds or thousands of variables. Working directly in such high-dimensional spaces has serious drawbacks, known as the **curse of dimensionality**: as the number of dimensions grows, the volume of the space grows exponentially, and any real dataset—however large—populates it extremely **sparsely**. Samples stop having truly close “neighbors”, and distances between points become nearly indistinguishable.

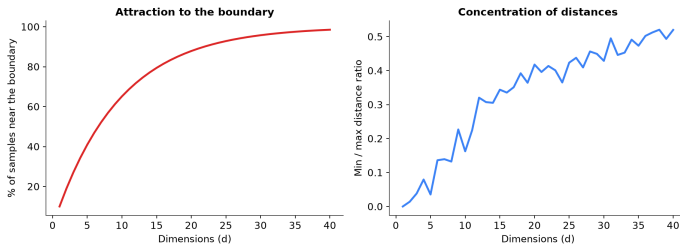


Why reduce dimensions?

Dimensionality reduction tackles this problem by projecting or compressing the data into a low-dimensional space (usually 2D or 3D). This offers several benefits:

- Drastically speeds up downstream computation.
- Mitigates the risk of overfitting.
- Is indispensable for **visualizing** complex data and extracting explainable latent factors.

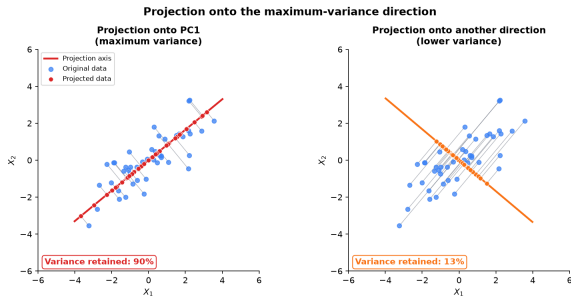
The curse of dimensionality, quantified



PCA: the direction of maximum variance

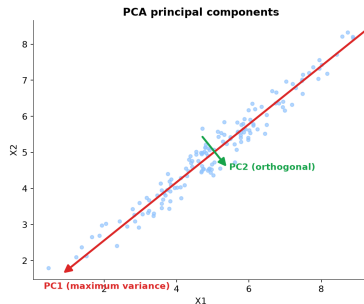
Principal Component Analysis (PCA) is the quintessential linear dimensionality-reduction algorithm. It orthogonally projects the original data onto a lower-dimensional subspace, minimizing information loss.

The **first principal component** (PC1) is the direction of the feature space along which the data **varies most**. Projecting the data onto this axis preserves as much of the original information as possible. The **second component** (PC2) seeks the direction explaining the most *remaining* variance, subject to being orthogonal to the first.



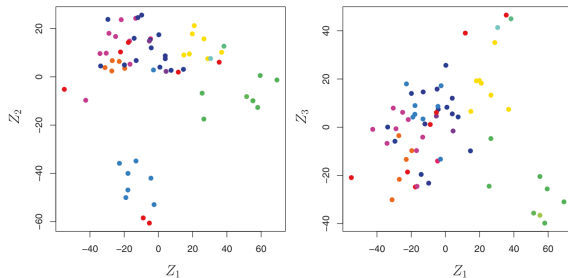
PCA: principal directions and reconstruction

PCA can be understood as a compression algorithm: an “encoder” reduces each sample to a smaller representation (its principal components), and a “decoder” reconstructs it back into the original space with minimal error. The fewer components kept, the greater the compression, but also the greater the information lost.



PCA on real data: NCI60 (6830 genes)

A real example of PCA's usefulness appears in the *NCI60* dataset, where each sample has 6830 genes (dimensions). Projecting these samples onto their first three principal components, cell lines of the same cancer type (same color) tend to cluster together, despite the huge original dimensionality.



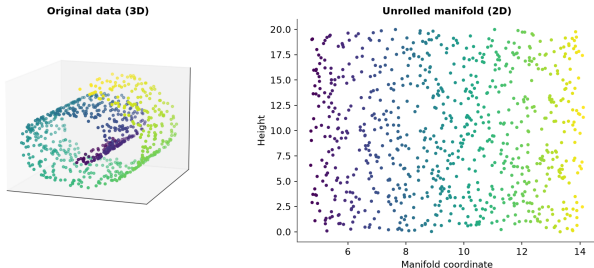
Source: James, Witten, Hastie & Tibshirani (2013), *An Introduction to Statistical Learning*, Fig. 10.15.

Manifold learning: when data isn't flat

PCA is fast and robust, but it assumes the interesting structure in the data is **linear**. When relationships are intrinsically non-linear (as in the classic Swiss roll example, a curved sheet of paper rolled in 3D), PCA's linear projection collapses points that are actually far apart.

The **manifold hypothesis** assumes that real-world data (images, voices, text) lives on a curved, low-dimensional surface (a **manifold**) embedded within the original high-dimensional space.

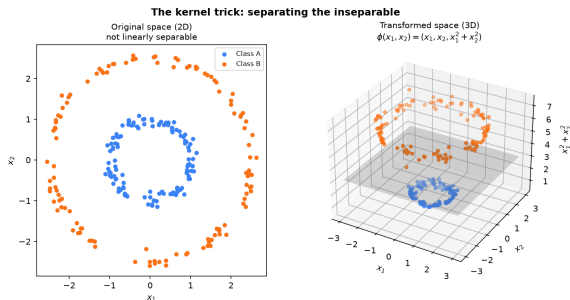
Swiss roll: the motivation for manifold learning



Kernel PCA: the kernel trick

To extend PCA to non-linear problems, **kernel PCA** implicitly maps the input data into a much higher-dimensional space, where relationships that were non-linear in the original space become **linearly separable**.

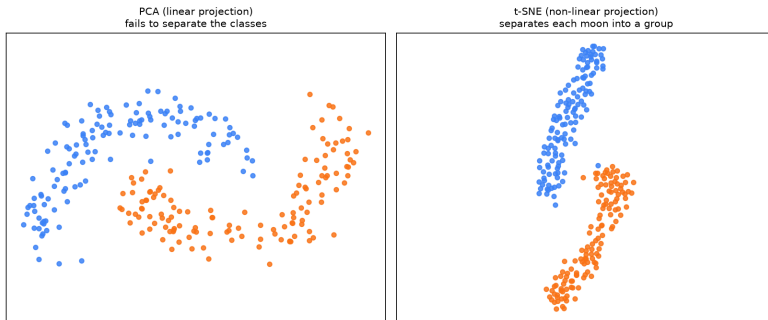
The classic example below illustrates this: two classes arranged in concentric circles (left) cannot be separated by a straight line in 2D; after applying the mapping, they sit at different heights and a simple plane separates them perfectly.



t -SNE: preserving local neighborhoods

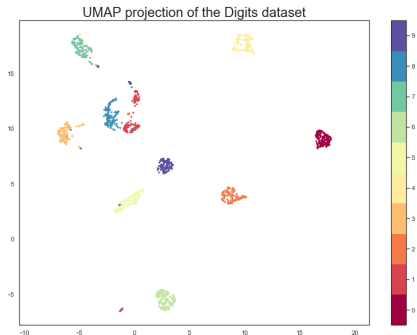
Proposed by Maaten and Hinton (2008), t -SNE converts distances between samples into **neighborhood probabilities**: nearby points should stay close after the 2D projection, and distant ones should stay distant. It is the preferred technique for the **qualitative visualization** of complex groupings, though it is slow on very large datasets.

PCA vs. t -SNE: two interleaved moons embedded in 30D



UMAP: fast and accurate

UMAP is one of the most powerful manifold learning techniques available today. Unlike t -SNE, which focuses almost exclusively on preserving very local neighborhoods, UMAP is able to preserve both **local** and **global** structure, and it also runs substantially faster on massive datasets with millions of samples.



Source: umap-learn docs, Copyright (c) 2017 Leland McInnes, BSD-3 license.

Outline

- 1 Introduction
- 2 3.1 Clustering
- 3 3.2 Dimensionality reduction
- 4 3.3 Anomaly detection**

Philosophy of anomaly detection

Anomaly detection is an unsupervised task of great industrial relevance: identifying unusual patterns or suspicious events within a stream of data. It is applied in financial fraud detection, network intrusion prevention, manufacturing quality control, and automated dataset cleaning.

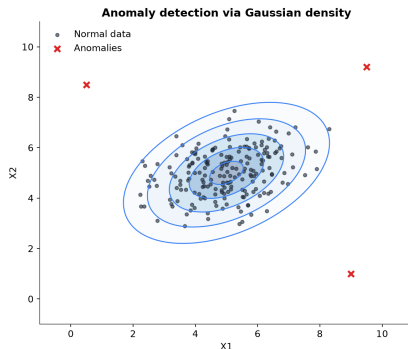
Conceptually, it rests on the premise that normal behaviors (*inliers*) are highly frequent and consistent, while anomalies (*outliers*) are rare events that deviate significantly from the norm. Two scenarios are distinguished based on how “clean” the training data is:

- **Anomaly detection:** the training set may contain unclean noise or outliers.
- **Novelty detection:** the training set is completely clean; the goal is to detect patterns never seen before.

Example: bank fraud, failures on a production line, or a sensor giving strange readings: in none of these cases do we know in advance what the next anomaly will look like, which is why this is treated as an unsupervised problem.

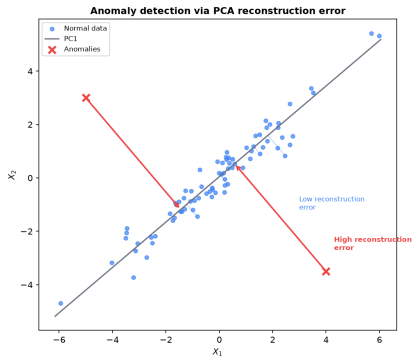
Gaussian approach

Normal observations are assumed to concentrate in regions of the feature space with **high probability density**. A Gaussian distribution is fit to the normal data, and any point with a probability density below a threshold is flagged as an anomaly. The threshold can be set, for example, from the known historical rate of rare cases (say, 4% defective products).



PCA reconstruction approach

This technique relies on the fact that PCA's principal components capture the directions characterizing the system's **normal** behavior. The data is projected to a low dimension with PCA and reconstructed back to the original space: since the principal components don't capture unusual deviations, anomalies will show a much larger **reconstruction error** than normal instances.

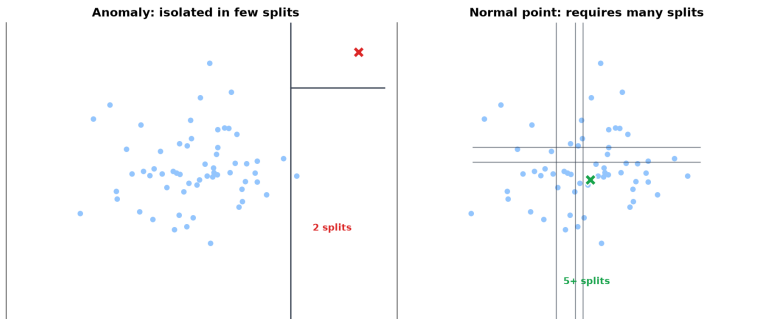


Isolation Forest

Unlike the previous methods, which model the density of normal data, **Isolation Forest** directly tries to **isolate** each observation through random partitions: at each node of a tree, a feature and a cutoff threshold are chosen at random, splitting the data in two.

The intuition is simple: anomalies, being far from the rest, require **very few partitions** to be isolated in their own leaf, while a normal point needs many more splits.

Isolation Forest: the isolation intuition

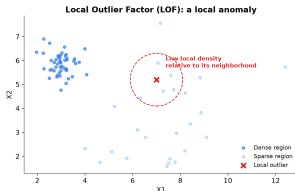


Local Outlier Factor (LOF)

LOF compares a sample's **local density** with that of its nearest neighbors. A normal instance has a density similar to its surroundings; a *local outlier* shows a much lower density than its immediate neighbors.

This method is very useful for detecting **local anomalies** that wouldn't stand out in a global analysis, because their absolute values aren't extreme, yet they are unusual within their specific neighborhood.

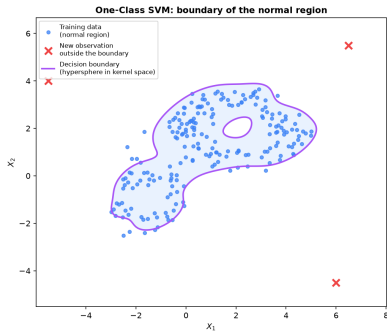
Example: a 200 m² house isn't unusual nationwide, but it is in a neighborhood where every house is 60 m²: that's exactly what LOF detects.



One-class SVM

This algorithm is optimized for novelty detection, when a clean training dataset is available. Instead of finding a hyperplane that separates two classes, it projects the data into a high-dimensional space via a kernel and looks for the **minimum-volume** region enclosing nearly all the training samples.

Any new observation falling outside that bounded region is automatically classified as an anomaly or novelty.



Chapter summary

- Unsupervised learning works **without labels**, discovering the latent structure of the data itself.
- **K-means** groups by centroids (fast, needs a known K); **hierarchical** clustering needs no K and gives a visual dendrogram; **DBSCAN** detects arbitrary shapes and isolates noise.
- The **silhouette coefficient** and the **gap statistic** are the most common tools for evaluating a clustering without labels.
- The **curse of dimensionality** motivates reducing the number of variables before analyzing or visualizing data.
- **PCA** is fast and linear; **kernel PCA**, **t -SNE**, and **UMAP** capture non-linear structure, with different trade-offs between speed and fidelity.
- Anomaly detection combines statistical approaches (Gaussian, PCA reconstruction) and specific algorithms (Isolation Forest, LOF, one-class SVM).

Chapter 3: Unsupervised Learning

Universidad de Salamanca

Faculty of Sciences

Advanced Programming

Diego M. Jiménez-Bravo · `dmjimenez@usal.es`